

HW2

Aakash,DA24B028

PART 2(MAC)

1.1

Before we even think about multiplying matrices, we need to make sure the dimensions match. Generally for $Y = W \cdot X + B$, the multiplication part is valid if the number of columns in W is the same as the number of rows in X , and the addition part is valid if the number of rows in B is the same as the number of rows in W and the number of columns in B is the same as the number of columns in X . Given that B is a column vector than X must be column vector and number of columns in W is same as number of rows in the X . In Hack VM we use a dedicated memory location (say `MEM[ERROR_FLAG]`) and set it to -1 on failure and 1 on success, then the caller can branch on that flag.

Listing 1: Dimension sanity check pseudocode

```
1 function sanityCheck(rW, cW, rX, cX, rB, cB):
2     // W * X is valid iff cW == rX
3     if cW != rX then
4         MEM[ERROR_FLAG] = -1
5         return false
6     endif
7
8     // for addition to be valid
9     if rB != rW or cB != cX then
10        MEM[ERROR_FLAG] = -1
11        return false
12    endif
13
14    // All good, signal success
15    MEM[ERROR_FLAG] = 1
16    return true
```

2.1

Approach A (MATMUL):

MATMUL is called exactly once. The call frame pushed onto the stack contains the return address, saved LCL, ARG, THIS, THAT, and a handful of pointer arguments, so the overhead is a flat, one-time cost of 5 words for bookkeeping plus whatever local variables the function declares. The nested loops live *inside* the function body, so the stack depth stays $O(1)$ (just this one frame) throughout the entire computation. No repeated frame push/pop in the hot path at all.

Approach B (ROWXCOL):

Here the caller drives the outer two loops and calls ROWXCOL for every single entry of the output matrix. Crucially, these calls happen sequentially ROWXCOL returns before the next call is made, so at any point in

time there are only two frames on the stack (the caller's + one ROWXCOL frame). Peak stack depth is therefore still $O(1)$.

The real cost is in **time**, not space. Each of the $m \cdot n$ calls pays the full Hack VM frame setup/teardown: pushing the return address, saving 5 registers, passing 3 arguments, then restoring everything on return and all calls giving a cumulative call overhead of $O(m \cdot n)$ in execution time.

2.2

Approach A

MATMUL is declared with 7 locals. The segment map is fixed for the entire lifetime of the function:

Segment	Slot	What lives there
argument 0	ARG[0]	pointer to W
argument 1	ARG[1]	pointer to X
argument 2	ARG[2]	pointer to B
argument 3	ARG[3]	m (rows of W)
argument 4	ARG[4]	p (cols of W = rows of X)
argument 5	ARG[5]	n (cols of X)
local 0	LCL[0]	loop counter i
local 1	LCL[1]	loop counter j
local 2	LCL[2]	loop counter k
local 3	LCL[3]	accumulator (dot product)
local 4	LCL[4]	current row pointer into W
local 5	LCL[5]	current col pointer into X
local 6	LCL[6]	write pointer into Y

The corresponding VM code for the innermost loop looks like:

Listing 2: Approach A – inner loop in VM pseudocode

```

1 // function MATMUL 7
2 // local 0=i, 1=j, 2=k, 3=acc, 4=rowPtr, 5=colPtr, 6=yPtr
3
4 label INNER_LOOP
5     // acc += W[rowPtr + k] * X[colPtr + k]
6     push local 4           // rowPtr
7     push local 2           // k
8     add                   // rowPtr + k (address of W element)
9     pop pointer 0          // THIS = rowPtr+k
10    push this 0            // W[i][k]
11
12    push local 5           // colPtr
13    push local 2           // k
14    add
15    pop pointer 1          // THAT = colPtr+k
16    push that 0            // X[k][j]
17
18    call Math.multiply 2
19    push local 3           // acc
20    add
21    pop local 3            // acc updated
22
23    push local 2
24    push constant 1
25    add
26    pop local 2            // k++

```

```

27
28     push local 2
29     push argument 4      // p
30     lt
31     if-goto INNER_LOOP

```

Every memory access is a simple `push local N` no save or restore, no segment pointer switching mid-computation.

Approach B

Now the work is split across two functions. The caller owns the loop structure and the subroutine owns only the dot product. Here's exactly what each function's segments hold:

Function	Segment	Slot	What lives there
caller	local 0	LCL[0]	loop counter i
caller	local 1	LCL[1]	loop counter j
caller	local 2	LCL[2]	write pointer into Y
caller	local 3	LCL[3]	base pointer of W
caller	local 4	LCL[4]	base pointer of X
ROWXCOL	argument 0	ARG[0]	row pointer (start of row i in W)
ROWXCOL	argument 1	ARG[1]	col pointer (start of col j in X)
ROWXCOL	argument 2	ARG[2]	length p
ROWXCOL	local 0	LCL[0]	loop counter k
ROWXCOL	local 1	LCL[1]	accumulator

The problem is what happens to the *caller's* LCL and ARG on each call. The VM `call` instruction overwrites LCL and ARG to point into the new frame. So the caller's i , j , `yPtr` etc. are temporarily unreachable – the VM saves them by pushing them onto the stack as part of the call frame, and restores them on `return`. This save/restore costs exactly 5 memory writes on `call` and 5 memory reads on `return`, every single iteration:

Listing 3: Approach B – per-iteration call overhead in generated assembly

```

1 // Before ROWXCOL call the VM translator emits:
2 push <return_address>    // 1 write
3 push LCL                 // 2 -- saves caller's local base
4 push ARG                 // 3 -- saves caller's arg base
5 push THIS                // 4
6 push THAT                // 5
7 ARG = SP - n_args - 5    // repoint ARG to new frame
8 LCL = SP                 // repoint LCL to new frame
9 goto ROWXCOL
10
11 // On return the VM translator emits:
12 THAT = *(LCL - 1)        // 5 reads to restore caller state
13 THIS = *(LCL - 2)
14 ARG = *(LCL - 3)
15 LCL = *(LCL - 4)
16 // jump back to return address

```

2.3

The real bottleneck in Approach A is the innermost loop — it performs $m \cdot n \cdot p$ multiply-accumulate operations (for multiplying an $m \times p$ matrix with a $p \times n$ one). So most optimisations should focus there. Some practical improvements:

1. **Loop unrolling.** Since each VM command translates to a Hack instruction, loop control (especially `if-goto`) adds noticeable overhead. Unrolling the inner loop by a factor like 4 or 8 reduces these checks and speeds things up.
2. **Pointer arithmetic over indexing.** Instead of recomputing `base + index` every iteration, just keep a running pointer and increment it. This replaces an expensive software multiply with a simple add — much cheaper on Hack.
3. **Transpose X .** If we transpose X beforehand, both the row of W and the column of X become contiguous in memory. This makes access sequential instead of strided, which is far more efficient given how Hack RAM is structured.
4. **Hardware support (if available).** Ideally, a multiply-accumulate (MAC) unit would collapse the entire inner loop into a single instruction. Even having a native multiply instruction would eliminate the need for a software multiplication loop.

2.4

The main issue with Approach B is the overhead from calling a function for every single output entry. That adds up quickly. Some compiler-level fixes:

1. **Inlining.** The simplest and most effective fix. Since `ROWXCOL` is small and used inside a tight loop, a compiler can inline it directly. This removes all the function call overhead and essentially reduces Approach B to something very close to A.
2. **Frame reuse or lightweight calls.** Even though calls aren't nested, the VM still rebuilds the stack frame every time. A smarter compiler could reuse the same frame and skip unnecessary saves/restores of `LCL`, `ARG`, `THIS`, and `THAT` when they don't change.
3. **Register-like use of `temp`.** Loop variables like i , j , and the output pointer are live across calls. Storing them in `temp` (which isn't preserved by the calling convention) avoids redundant save/restore overhead compared to `local`.
4. **Batching calls.** Instead of calling `ROWXCOL` for each element, process multiple elements at once when they share the same row (fixed i , varying j). This spreads the call overhead across several outputs and improves efficiency.

Extra credit

My favourite song is Aaya-sher